

TRABAJO PRÁCTICO INTEGRADOR: INFORME TÉCNICO

Materia: Programación 1

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Resistencia

Tecnicatura Universitaria en Programación – Modalidad a Distancia

Asignatura: Programación 1

Proyecto: Sistema de Gestión de Datos de Países: Estructuras Basadas en Colecciones, Algoritmia Manual y Persistencia en Archivos Planos

Docentes: Oscar Londero - Sofia Elizabeth Fernandez

Integrantes:

- Fernández, Martín
- Fernández, Facundo

Año: 2026



Contenido

TRABAJO PRÁCTICO INTEGRADOR: INFORME TÉCNICO	1
Materia: Programación 1	1
1. Introducción al Proyecto.....	3
2. Marco Teórico (Parte I: Estructuras de Datos y Control).....	3
2.1 Estructuras de Datos Lineales y Colecciones	3
2.2 Modularización mediante Funciones (def)	3
2.3 Estructuras Cíclicas y Lógica de Robustez	4
2.4 Persistencia de Datos en Archivos Planos (CSV).....	4
2.5 Algoritmo de Ordenamiento Manual (Bubble Sort)	4
2.6 Análisis Estadístico e Indicadores Analíticos Manuales.....	5
3. Evidencia de Funcionamiento (Capturas de Pantalla)	6
4. Dificultades Encontradas y Soluciones Implementadas	8
5. Conclusiones.....	9

1. Introducción al Proyecto

El presente informe documenta el diseño, la arquitectura y la implementación de una aplicación de software orientada a la gestión y análisis analítico de datos geopolíticos de países. Desarrollado en el lenguaje de programación Python, el sistema aborda de manera integral problemas clásicos de la ingeniería de software: el procesamiento en memoria de colecciones de datos dinámicas, la persistencia de información en almacenamiento secundario mediante archivos CSV, la implementación manual de algoritmos de ordenamiento y la optimización de la robustez mediante técnicas avanzadas de validación de entradas.

El objetivo principal es proveer una herramienta eficiente en consola que permita realizar operaciones CRUD (Alta y Modificación), búsquedas indexadas y generación de reportes estadísticos globales, operando bajo restricciones académicas estrictas que prohíben el uso de librerías externas o funciones de alto nivel preconstruidas en el lenguaje.

2. Marco Teórico (Parte I: Estructuras de Datos y Control)

2.1 Estructuras de Datos Lineales y Colecciones

Para la manipulación eficiente del dataset en memoria de trabajo (RAM), se optó por un enfoque híbrido combinando las dos estructuras nativas más potentes de Python:

- **Listas (`list`):** Definidas como colecciones ordenadas, mutables y de tamaño dinámico. En este proyecto, la lista actúa como el contenedor maestro o base de datos temporal que almacena la totalidad de los registros de países. Su elección se justifica en la necesidad de iterar secuencialmente sobre los elementos y aplicar intercambios de posición físicos durante la ejecución del algoritmo de ordenamiento.
- **Diccionarios (`dict`):** Estructuras asociativas no ordenadas que almacenan elementos bajo la relación clave-valor. Cada país se modela como un diccionario independiente, donde las claves del sistema (`'nombre'`, `'continente'`, `'poblacion'`, `'superficie'`) actúan como los campos de un registro tradicional. Esto optimiza la legibilidad del código fuente, permitiendo acceder a atributos específicos mediante etiquetas semánticas en lugar de índices numéricos fijos.

2.2 Modularización mediante Funciones (`def`)

La arquitectura del software se rige por el paradigma de descomposición funcional. Cada requerimiento del menú principal se aisló en funciones independientes con responsabilidades únicas y bien definidas.

Las funciones diseñadas implementan el paso de parámetros por referencia (debido al comportamiento mutable de las listas y diccionarios en Python), lo que permite que las modificaciones realizadas en los módulos de alta o edición se reflejen inmediatamente en la estructura central del programa sin necesidad de duplicar datos en memoria.

2.3 Estructuras Cíclicas y Lógica de Robustez

El flujo del programa está gobernado por bucles de control controlados por condiciones lógicas:

- **Ciclos Indefinidos (`while`):** Utilizados para mantener el menú interactivo activo de forma persistente hasta que el usuario decida explícitamente egresar (opción 0). Asimismo, representan la columna vertebral del sistema de validación de datos: ante un ingreso erróneo por consola, un bucle `while` atrapa el flujo del programa e impide el avance hasta que la entrada cumpla estrictamente con las restricciones de tipo (numérico, no vacío, rango válido), garantizando la inmunidad del programa ante fallas de ejecución en tiempo de diseño.
- **Ciclos Definidos (`for`):** Aplicados exhaustivamente para la lectura secuencial de estructuras de datos y para la implementación del algoritmo de ordenamiento por burbuja.

2.4 Persistencia de Datos en Archivos Planos (CSV)

Para garantizar la persistencia de la información más allá del ciclo de vida de la ejecución del programa, el sistema utiliza archivos externos en formato CSV (Comma-Separated Values / Valores Separados por Comas). Los archivos CSV representan un estándar en la manipulación de datos debido a su simplicidad estructurada, legibilidad y alta interoperabilidad con herramientas de análisis de datos y hojas de cálculo.

- **Lectura y Parsing Secuencial:** El proceso de carga transforma cadenas de texto planas en estructuras de datos estructuradas en memoria. Para ello, se lee el archivo línea por línea omitiendo el encabezado. Se aplica el método `.strip()` para limpiar caracteres de control ocultos (como saltos de línea `\n`) y el método `.split(",")` para segmentar la cadena en campos individuales utilizando la coma como delimitador.
- **Manejo de Excepciones y Robustez:** Con el fin de evitar que el programa aborte su ejecución ante fallas imprevistas, se implementa un bloque de control de excepciones `try-except` capturando de forma selectiva el error `FileNotFoundError`. Si el archivo de datos no se encuentra en el directorio raíz, el programa intercepta el error, notifica al usuario y genera de manera automatizada un archivo base con sus respectivos encabezados. Esto asegura que la aplicación pueda iniciar desde cero con una lista vacía sin interrumpir la experiencia del usuario.
- **Validación de Tipos de Datos:** Durante el proceso de lectura, se verifica algorítmicamente mediante el método `.isdigit()` que las cadenas destinadas a representar la población y la superficie contengan únicamente caracteres numéricos antes de forzar su conversión a tipo entero (`int`). Si una línea no cumple con la estructura requerida (por poseer campos vacíos o tipos de datos incorrectos), el registro es omitido informando la línea exacta del error, manteniendo la integridad de la base de datos local.

2.5 Algoritmo de Ordenamiento Manual (Bubble Sort)

El ordenamiento de la información en memoria es un requerimiento crítico para el análisis de datos. En cumplimiento estricto con las directrices académicas de la cátedra, se descartó el

uso de funciones integradas de Python (como `sort()` o `sorted()`) y se implementó de forma manual el algoritmo de ordenamiento por burbuja (**Bubble Sort**).

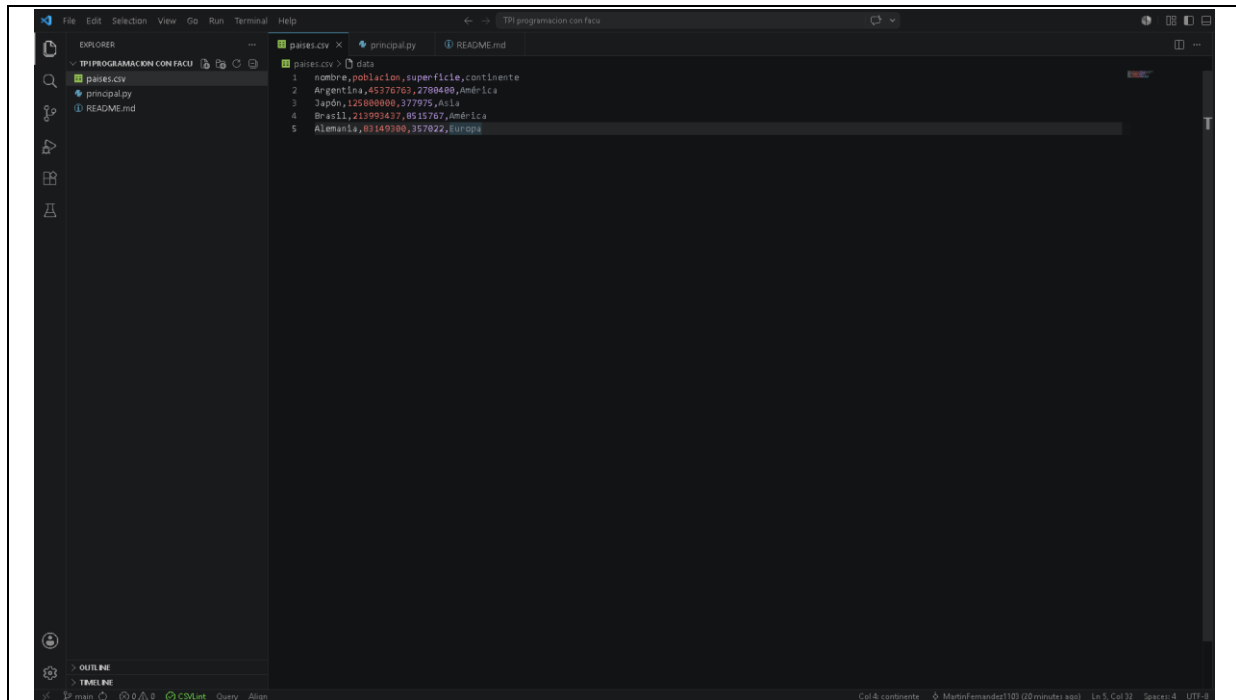
- **Mecanismo de Funcionamiento:** Este algoritmo opera realizando múltiples pasadas consecutivas sobre la lista de países. En cada iteración, compara los valores de una clave específica (nombre, población o superficie) de dos elementos adyacentes (`j` y `j+1`). Si los elementos se encuentran en un orden incorrecto según el criterio seleccionado (ascendente o descendente), se realiza un intercambio físico de sus posiciones en la lista.
- **Complejidad Computacional:** El algoritmo utiliza dos bucles anidados (`for`), lo que le otorga una complejidad temporal en el peor de los casos de $O(n^2)$, donde n es la cantidad de países registrados. A pesar de no ser el algoritmo más eficiente para datasets masivos, su implementación es de alto valor pedagógico para demostrar el dominio de los índices, punteros y estructuras lógicas básicas.
- **Flexibilidad de Criterios:** Para lograr un ordenamiento robusto, el código evalúa de manera dinámica las claves internas de los diccionarios. En el caso específico del campo `'nombre'`, se aplica el método `.lower()` de forma temporal durante la comparación para asegurar un ordenamiento alfabético insensible a mayúsculas y minúsculas, evitando las discrepancias que genera el código ASCII estándar.

2.6 Análisis Estadístico e Indicadores Analíticos Manuales

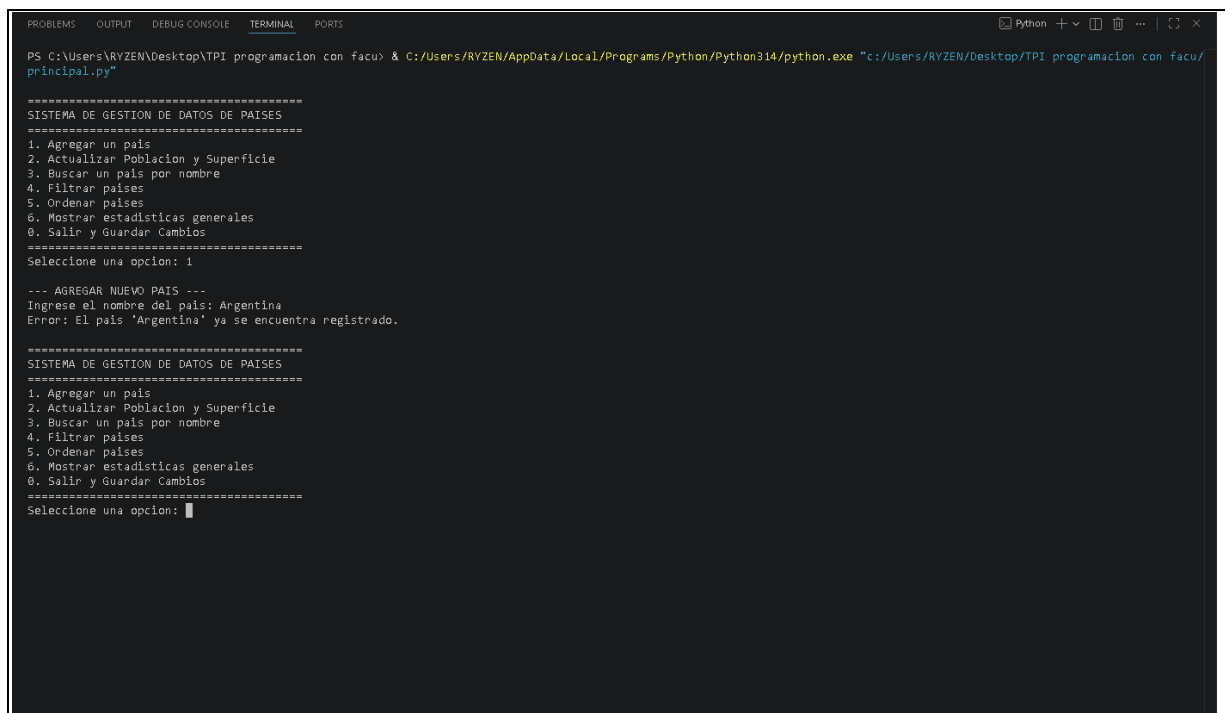
La generación de reportes analíticos representa la fase final del procesamiento de datos del sistema. De la misma manera que en el módulo de ordenamiento, el cálculo de métricas e indicadores globales se diseñó omitiendo el uso de funciones de agregación de alto nivel como `sum()`, `max()` o `min()`.

- **Acumuladores Basales:** El cálculo de la población total global y la superficie total acumulada se efectúa mediante un único recorrido secuencial (`for`) sobre la lista de países. En cada ciclo, los valores correspondientes se adicionan a variables acumuladoras inicializadas en cero. Los promedios se determinan dividiendo estos totales acumulados por la longitud de la lista, calculada de forma dinámica.
- **Determinación de Valores Extremos (Máximos y Mínimos):** Para identificar los países con las mayores y menores métricas de población y superficie sin utilizar funciones del sistema, se utiliza la técnica de inicialización de extremos en base al primer elemento del vector. Al comenzar el análisis, se asume que el primer país de la lista (`lista_paises[0]`) posee simultáneamente el valor máximo y el mínimo. A medida que el bucle recorre los registros restantes, se evalúan condicionales independientes: si el país actual supera al máximo registrado, la variable toma el diccionario actual; si es menor al mínimo registrado, se actualiza la referencia. Esto permite resolver todas las consultas analíticas en un único recorrido lineal de complejidad, optimizando el uso de la memoria RAM.

3. Evidencia de Funcionamiento (Capturas de Pantalla)



Entorno de Desarrollo en Visual Studio Code. Evidencia del espacio de trabajo estructurado con los archivos principal.py, países.csv y README.md correctamente sincronizados.



Menú Principal y Alta de Registros. Demostración de la interfaz en consola interceptando campos vacíos y duplicados de nombres de países.

```
PS C:\Users\RYZEN\Desktop\TPI programacion con facu> & C:/Users/RYZEN/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/RYZEN/Desktop/TPI programacion con facu/principal.py"

=====
SISTEMA DE GESTION DE DATOS DE PAISES
=====
1. Agregar un pais
2. Actualizar Poblacion y Superficie
3. Buscar un pais por nombre
4. Filtrar paises
5. Ordenar paises
6. Mostrar estadísticas generales
0. Salir y Guardar Cambios
=====
Seleccione una opcion: 5

--- MODULO DE ORDENAMIENTO MANUAL ---
1. Ordenar por Nombre
2. Ordenar por Cantidad de Poblacion
3. Ordenar por Superficie (km2)
Seleccione el criterio de ordenamiento: 1

¿En que sentido desea ordenar?
1. Ascendente (Menor a Mayor / A-Z)
2. Descendente (Mayor a Menor / Z-A)
Seleccione el sentido: 1

Lista ordenada por [NOMBRE] de forma ASCENDENTE:
=====
Alemania      | Continente: Europa   | Poblacion: 83149300 | Superficie: 357022 km2
Argentina     | Continente: América  | Poblacion: 45376763 | Superficie: 2780400 km2
Brasil        | Continente: América  | Poblacion: 213993437 | Superficie: 8515767 km2
Japón         | Continente: Asia     | Poblacion: 125800000 | Superficie: 377975 km2
=====

SISTEMA DE GESTION DE DATOS DE PAISES
=====
1. Agregar un pais
2. Actualizar Poblacion y Superficie
3. Buscar un pais por nombre
4. Filtrar paises
5. Ordenar paises
6. Mostrar estadísticas generales
0. Salir y Guardar Cambios
=====
Seleccione una opcion: []
```

Ejecución del Módulo de Ordenamiento Manual. Pantallazo de la terminal ejecutando el algoritmo Bubble Sort aplicado de forma alfabética ascendente por el campo nombre.

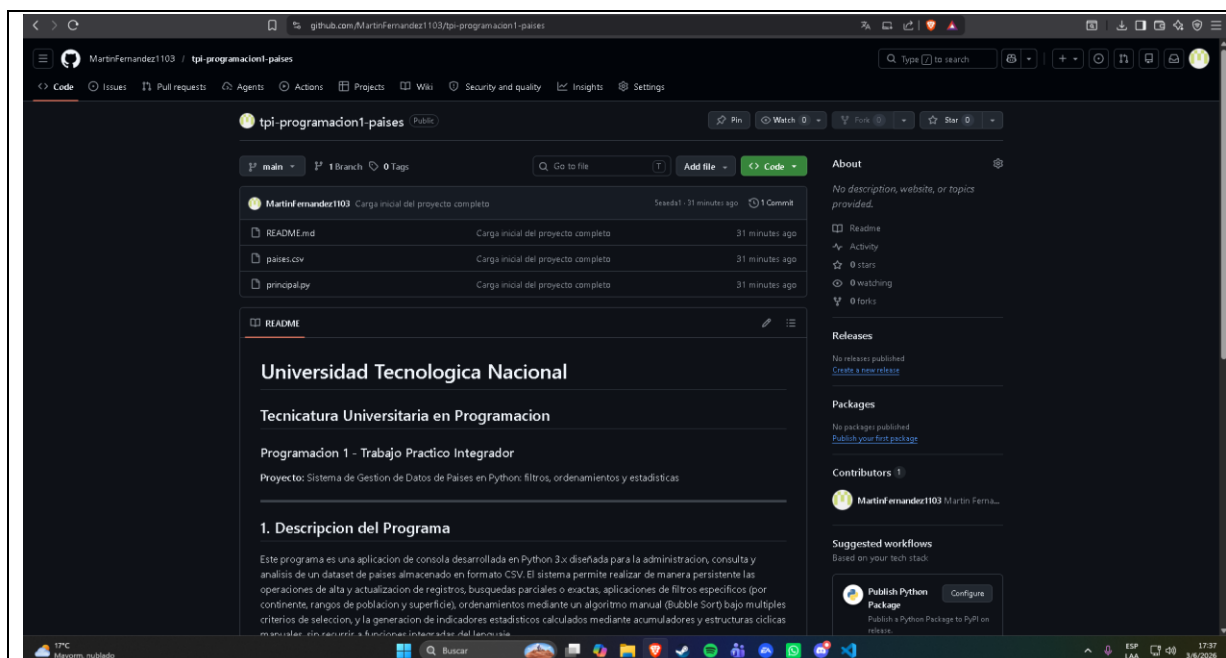
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\RYZEN\Desktop\TPI programacion con facu> & C:/Users/RYZEN/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/RYZEN/Desktop/TPI programacion con facu/principal.py"

=====
SISTEMA DE GESTION DE DATOS DE PAISES
=====
1. Agregar un pais
2. Actualizar Poblacion y Superficie
3. Buscar un pais por nombre
4. Filtrar paises
5. Ordenar paises
6. Mostrar estadísticas generales
0. Salir y Guardar Cambios
=====
Seleccione una opcion: 6

--- MODULO DE ESTADISTICAS GENERALES ---
Cantidad total de paises registrados: 4
=====
INDICADORES DE POBLACION:
Poblacion total global      : 468319500 habitantes
Promedio de poblacion x pais: 117079875.00 habitantes
Pais mas poblado             : Brasil (213993437 habitantes)
Pais menos poblado           : Argentina (45376763 habitantes)
=====
INDICADORES DE SUPERFICIE:
Superficie total global     : 12831164 km2
Promedio de superficie x pais: 3897791.00 km2
Pais con mayor superficie   : Brasil (8515767 km2)
Pais con menor superficie    : Alemania (357022 km2)
=====

SISTEMA DE GESTION DE DATOS DE PAISES
=====
1. Agregar un pais
2. Actualizar Poblacion y Superficie
3. Buscar un pais por nombre
4. Filtrar paises
5. Ordenar paises
6. Mostrar estadísticas generales
0. Salir y Guardar Cambios
=====
Seleccione una opcion: []
```

Módulo de Estadísticas Generales. Muestra del reporte analítico en pantalla con los promedios calculados y la identificación precisa de los países extremos (máximos y mínimos).



Sincronización con Repositorio Remoto en GitHub. Evidencia del repositorio público en la plataforma web con la confirmación de la carga exitosa de todos los elementos del proyecto a través de Git Bash.

4. Dificultades Encontradas y Soluciones Implementadas

- A. Validación de Tipos Numéricos sin Estructuras de Excepción: La restricción metodológica de no utilizar bloques try-except para validar las entradas de la consola representó un desafío de diseño en los submenús de rangos y cargas numéricas. Esta dificultad se solucionó mediante el uso exhaustivo del método string .isdigit() combinado con bucles indefinidos while True, logrando filtrar de manera lógica cadenas alfabéticas y entradas vacías antes de intentar la conversión de tipo.
- B. Persistencia Transparentar ante Archivos Inexistentes: Durante el despliegue del sistema, la ausencia del archivo paises.csv provocaba la interrupción del programa por excepciones en tiempo de ejecución. Para resolverlo, se diseñó un bloque de manejo preventivo que captura el error FileNotFoundError, emite una alerta no bloqueante y genera automáticamente una plantilla limpia con los encabezados correspondientes, asegurando la resiliencia de la aplicación.
- C. Lógica de Extremos Dinámicos: Evitar el uso de funciones de agregación como max o min requirió un diseño preciso en el manejo de punteros a diccionarios. Inicializar los extremos referenciando al índice cero de la lista impidió errores de lógica clásicos

(como inicializar mínimos en cero), asegurando que el sistema responda correctamente independientemente de la magnitud de los valores del dataset.

5. Conclusiones

El desarrollo de este sistema de gestión integrado permitió consolidar de manera práctica las competencias analíticas y algorítmicas promovidas por la cátedra de Programación 1. La implementación de operaciones complejas sobre estructuras de datos dinámicas, como listas de diccionarios, demostró la flexibilidad que posee Python para el modelado de entidades del mundo real sin necesidad de recurrir a bases de datos relacionales robustas en etapas tempranas del diseño de software.

Asimismo, la construcción manual de algoritmos de ordenamiento y computación estadística reforzó la comprensión interna de los ciclos de control, la manipulación física de posiciones en memoria y la gestión de la complejidad computacional. Finalmente, la integración del control de versiones con Git y la automatización de la persistencia en formato CSV aproximan el ejercicio académico a los estándares profesionales de la industria, garantizando la entrega de un producto de software robusto, documentado y completamente funcional.